Materiały dydaktyczne przeznaczone wyłącznie dla studentów wydziału Informatyki i Telekomunikacji

Przesyłanie danych multimedialnych

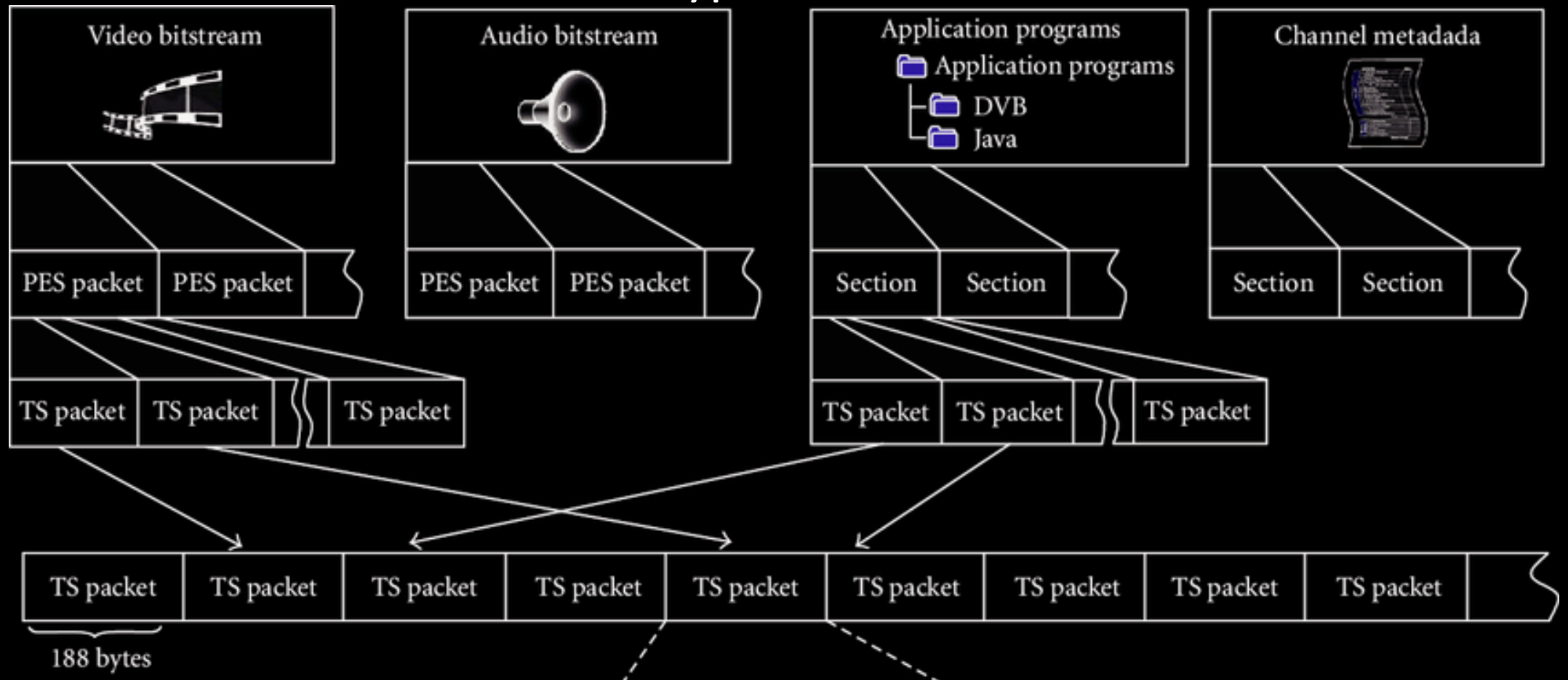
MPEG-TS - Transport stream

Packetized Elementary Stream

Jakub Stankowski

Institute of Multimedia Telecommunications · Faculty of Computing and Telecommunications
Poznan University of Technology

Przypomnienie



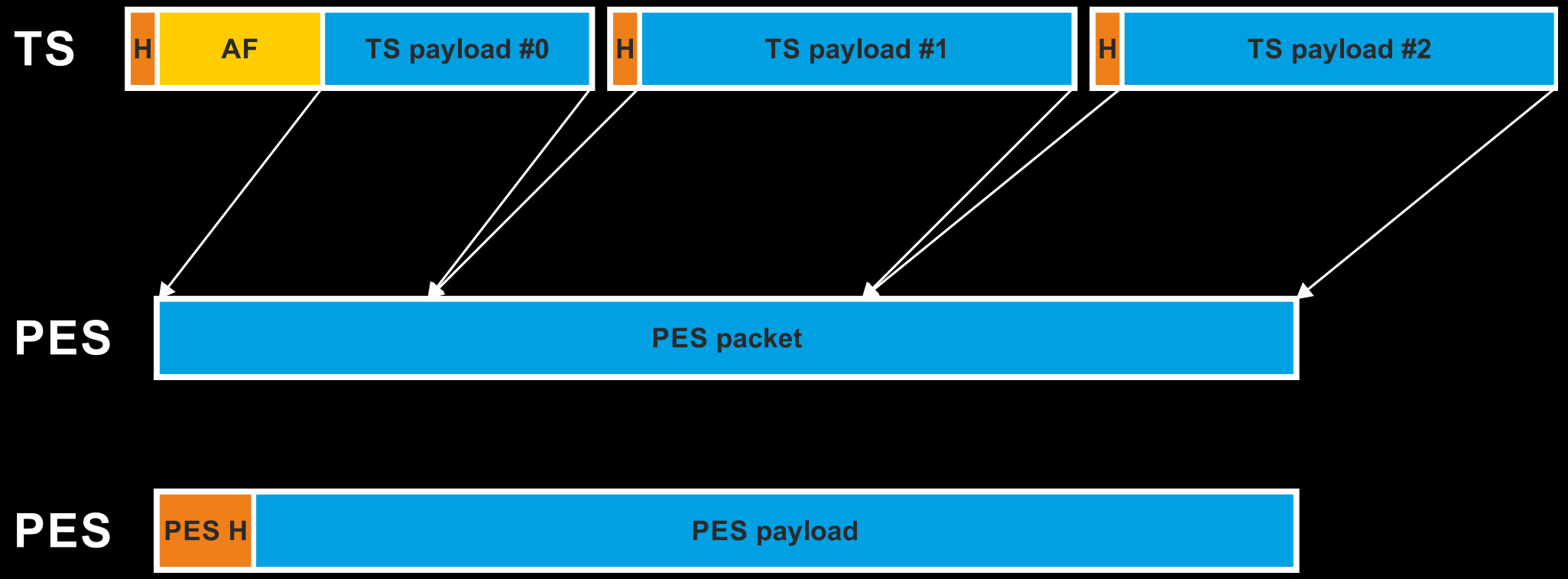
Field	Sync. byte	Transport error indicator	Payload unit start indicator	Transport priority	Packet ID (PID)	Transport scrambling control	Adaptation field control	Continuity count
Number of bits	8	1	1	1	13	2	2	4

Przykład

```
0000000000 TS: SB=71 E=0 S=1 P=0 PID=  17 TSC=0 AF=1 CC= 0
0000000001 TS: SB=71 E=0 S=1 P=0 PID=   0 TSC=0 AF=1 CC= 0
0000000002 TS: SB=71 E=0 S=1 P=0 PID=4096 TSC=0 AF=1 CC= 0
0000000003 TS: SB=71 E=0 S=1 P=0 PID= 136 TSC=0 AF=3 CC= 0 AF: L=  1 DC=0 RA=1 SP=0 PR=0 OR=0 SP=0 TP=0 EX=0
0000000004 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC= 1
0000000005 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC= 2
0000000006 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC= 3
0000000007 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC= 4
0000000008 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC= 5
0000000009 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC= 6
0000000010 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC= 7
0000000011 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC= 8
0000000012 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC= 9
0000000013 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC=10
0000000014 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC=11
0000000015 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC=12
0000000016 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC=13
0000000017 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC=14
0000000018 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=3 CC=15 AF: L= 47 DC=0 RA=0 SP=0 PR=0 OR=0 SP=0 TP=0 EX=0
0000000019 TS: SB=71 E=0 S=1 P=0 PID= 136 TSC=0 AF=3 CC= 0 AF: L=  1 DC=0 RA=1 SP=0 PR=0 OR=0 SP=0 TP=0 EX=0
0000000020 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC= 1
0000000021 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC= 2
0000000022 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC= 3
0000000023 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC= 4
0000000024 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC= 5
0000000025 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC= 6
0000000026 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC= 7
0000000027 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC= 8
0000000028 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC= 9
0000000029 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC=10
0000000030 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC=11
0000000031 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC=12
0000000032 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC=13
0000000033 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC=14
```

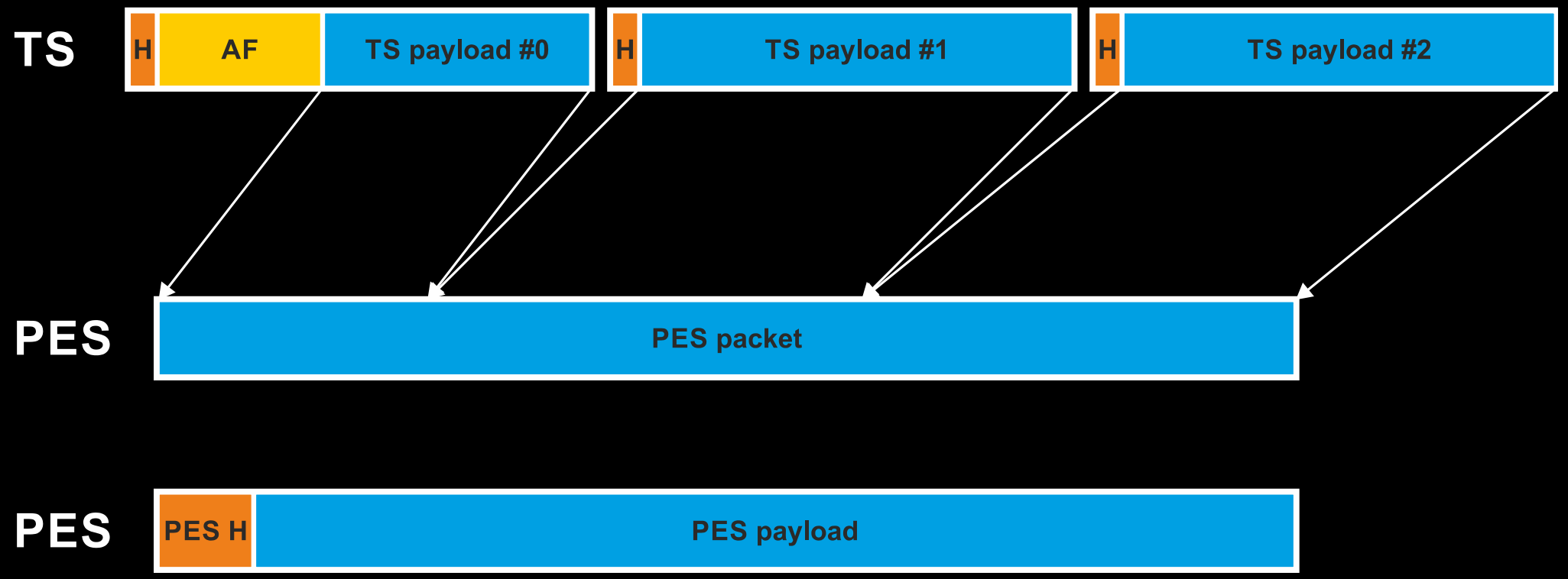
Packetized Elementary Stream

- Strumień elementarny – pakiet
  - header (nagłówek)
    - zmienna długość nagłówka (>=6 B)
  - payload (pakiet danych, np. ramka fonii)
  - ITU-T H.222.0 | ISO/IEC 13818-1
    - Section 2.4.3.6 – PES packet



Packetized Elementary Stream

- Strumień elementarny – nagłówek
  - podstawowe pola (pierwsze 6B)
    - `packet_start_code_prefix (24) = 0x000001`
    - `stream_id ( 8)`
    - `PES_packet_length (16)`
  - `PES_packet_length` może przyjmować wartość 0



Packetized Elementary Stream - Header

Table 2-21 – PES packet

Syntax	No. of bits	Mnemonic
PES_packet() {		
packet_start_code_prefix	**24**	**bslbf**
stream_id	**8**	**uimsbf**
PES_packet_length	**16**	**uimsbf**
if (stream_id != program_stream_map		
&& stream_id != padding_stream		
&& stream_id != private_stream_2		
&& stream_id != ECM		
&& stream_id != EMM		
&& stream_id != program_stream_directory		
&& stream_id != DSMCC_stream		
&& stream_id != ITU-T Rec. H.222.1 type E stream) {		
'10'	**2**	**bslbf**
PES_scrambling_control	**2**	**bslbf**
PES_priority	**1**	**bslbf**
data_alignment_indicator	**1**	**bslbf**
copyright	**1**	**bslbf**
original_or_copy	**1**	**bslbf**
PTS_DTS_flags	**2**	**bslbf**
ESCR_flag	**1**	**bslbf**
ES_rate_flag	**1**	**bslbf**
DSM_trick_mode_flag	**1**	**bslbf**
additional_copy_info_flag	**1**	**bslbf**
PES_CRC_flag	**1**	**bslbf**
PES_extension_flag	**1**	**bslbf**
PES_header_data_length	**8**	**uimsbf**

Parser strumienia transportowego

- Krok 3:
  - pobranie i otwarcie dokumentu ITU-T H.222.0 | ISO/IEC 13818-1
  - „składanie” pakietu strumienia elementarnego
    - dla wybranej wartości PID=136 (fonia)
    - parsowanie nagłówka pakietu PES
    - wyliczenie długości nagłówka
    - weryfikacja poprawności
      - prawidłowy PID, ciągłość CC
      - porównanie:
        - » wartości pola `PES_packet_length`
        - » wyliczonej długości nagłówka pakietu PES (włączając opcjonalne pole `PES_header_data_length`
        - » długości pakietu

- Zadanie dodatkowe 2:
  - parsowanie zawartości pól PTS i DTS

Przykład

Adaptation field

```
0000000003 TS: SB=71 E=0 S=1 P=0 PID= 136 TSC=0 AF=3 CC= 0 AF: L=  1 DC=0 RA=1 SP=0 PR=0 OR=0 SP=0 TP=0 EX=0 Stuffing=0
0000000018 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=3 CC=15 AF: L= 47 DC=0 RA=0 SP=0 PR=0 OR=0 SP=0 TP=0 EX=0 Stuffing=46
0000000019 TS: SB=71 E=0 S=1 P=0 PID= 136 TSC=0 AF=3 CC= 0 AF: L=  1 DC=0 RA=1 SP=0 PR=0 OR=0 SP=0 TP=0 EX=0 Stuffing=0
```

Packetized Elementary Stream

```
0000000003 TS: SB=71 E=0 S=1 P=0 PID= 136 TSC=0 AF=3 CC= 0 Started  PES: PSCP=1 SID=192 L=2888
0000000004 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC= 1 Continue
0000000005 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC= 2 Continue
0000000006 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC= 3 Continue
0000000007 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC= 4 Continue
0000000008 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC= 5 Continue
0000000009 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC= 6 Continue
0000000010 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC= 7 Continue
0000000011 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC= 8 Continue
0000000012 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC= 9 Continue
0000000013 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC=10 Continue
0000000014 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC=11 Continue
0000000015 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC=12 Continue
0000000016 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC=13 Continue
0000000017 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=1 CC=14 Continue
0000000018 TS: SB=71 E=0 S=0 P=0 PID= 136 TSC=0 AF=3 CC=15 Finished PES: Len=2894
0000000019 TS: SB=71 E=0 S=1 P=0 PID= 136 TSC=0 AF=3 CC= 0 Started  PES: PSCP=1 SID=192 L=2888
```

Szablon klasy xPES_PacketHeader

```
class xPES_PacketHeader
{
public:
  enum eStreamId : uint8_t
  {
    eStreamId_program_stream_map       = 0xBC,
    eStreamId_padding_stream           = 0xBE,
    eStreamId_private_stream_2         = 0xBF,
    eStreamId_ECM                      = 0xF0,
    eStreamId_EMM                      = 0xF1,
    eStreamId_program_stream_directory = 0xFF,
    eStreamId_DSMCC_stream             = 0xF2,
    eStreamId_ITUT_H222_1_type_E       = 0xF8,
  };

protected:
  //PES packet header
  uint32_t m_PacketStartCodePrefix;
  uint8_t  m_StreamId;
  uint16_t m_PacketLength;

public:
  void     Reset();
  int32_t  Parse(const uint8_t* Input);
  void     Print() const;

public:
  //PES packet header
  uint32_t getPacketStartCodePrefix() const { return m_PacketStartCodePrefix; }
  uint8_t  getStreamId             () const { return m_StreamId; }
  uint16_t getPacketLength         () const { return m_PacketLength; }
};
```

Szablon klasy xPES_Assembler

```
class xPES_Assembler
{
public:
  enum class eResult : int32_t
  {
    UnexpectedPID     = 1,
    StreamPackedLost  ,
    AssemblingStarted ,
    AssemblingContinue,
    AssemblingFinished,
  };

protected:
  //setup
  int32_t  m_PID;
  //buffer
  uint8_t* m_Buffer;
  uint32_t m_BufferSize;
  uint32_t m_DataOffset;
  //operation
  int8_t            m_LastContinuityCounter;
  bool              m_Started;
  xPES_PacketHeader m_PESH;

public:
  xPES_Assembler ();
  ~xPES_Assembler();

  void    Init        (int32_t PID);
  eResult AbsorbPacket(const uint8_t* TransportStreamPacket, const xTS_PacketHeader* PacketHeader, const xTS_AdaptationField* AdaptationField);

  void     PrintPESH        () const { m_PESH.Print(); }
  uint8_t* getPacket        ()       { return m_Buffer; }
  int32_t  getNumPacketBytes() const { return m_DataOffset; }

protected:
  void xBufferReset ();
  void xBufferAppend(const uint8_t* Data, int32_t Size);
};
```

Przykładowy kod w funkcji `main`

```
size_t NumRead = fread(TS_PacketBuffer, 1, xTS::TS_PacketLength, TransportStreamFile);
if(NumRead != xTS::TS_PacketLength) { break; }

TS_PacketHeader.Reset();
TS_PacketHeader.Parse(TS_PacketBuffer);

TS_PacketAdaptationField.Reset();
if(TS_PacketHeader.getSyncByte() == 'G' && TS_PacketHeader.getPID() == 136)
{
  if(TS_PacketHeader.hasAdaptationField())
  {
    TS_PacketAdaptationField.Parse(TS_PacketBuffer, TS_PacketHeader.getAdaptationFieldControl());
  }

  printf("%010d ", TS_PacketId);
  TS_PacketHeader.Print();
  if(TS_PacketHeader.hasAdaptationField()) { TS_PacketAdaptationField.Print(); }

  xPES_Assembler::eResult Result = PES_Assembler.AbsorbPacket(TS_PacketBuffer, &TS_PacketHeader, &TS_PacketAdaptationField);
  switch(Result)
  {
    case xPES_Assembler::eResult::StreamPackedLost  : printf("PcktLost "); break;
    case xPES_Assembler::eResult::AssemblingStarted : printf("Started  "); PES_Assembler.PrintPESH(); break;
    case xPES_Assembler::eResult::AssemblingContinue: printf("Continue "); break;
    case xPES_Assembler::eResult::AssemblingFinished: printf("Finished "); printf("PES: Len=%d", PES_Assembler.getNumPacketBytes()); break;
    default: break;
  }
  printf("\n");
}
```